

A. Assignment Information

Field	Details
Department	Computer Science and Engineering
Programme	B.E. / B.Tech – CSE
Course Code & Course Name	CSA51 – Cryptography and Network Security
Academic Year / Batch	2026 – 2027
Assignment Number	Assignment 4
Assignment Title	Design and Implementation of a Secure Digital Certificate Management System
Course Outcome(s) – CO	CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. CO4: Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email security. CO5: Develop network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.
Bloom's Taxonomy Level	BL5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 9 – Industry, Innovation and Infrastructure; SDG 16 – Peace, Justice and Strong Institutions; SDG 17 – Partnerships for the Goals
Industry / Societal Relevance	X.509-based PKI underlies TLS/HTTPS, VPNs, code signing, and enterprise single sign-on used across virtually every industry sector.

B. Assignment Problem / Challenge

This assignment requires students to apply core Cryptography and Network Security concepts — cryptographic hashing, digital signatures, and X.509 authentication frameworks (CO3); secure protocol implementation for web/email services (CO4); and network-security strategy including protection against impersonation and malicious actors (CO5) — to a realistic enterprise identity-and-trust problem. Students must analyse the given scenario, design a Public Key Infrastructure (PKI), build a certificate-management simulator to validate the design, and evaluate the resulting framework against defined security criteria.

C. Problem Statement

Scenario: Secure Digital Certificate Management for a Multinational Organization

A multinational organization needs to securely authenticate its employees, servers, and partner organizations while exchanging sensitive information over web and application services. The organization wants to eliminate unauthorized access and impersonation attacks by establishing a centralized Public Key Infrastructure (PKI).

As a PKI Security Analyst, you are asked to examine and design a certificate-management framework using X.509 digital certificates. Your solution should implement and demonstrate:

- Generation of public/private key pairs for the CA and for entities (server, client/employee).

- Certificate Signing Request (CSR) creation by an entity.
- Certificate issuance by a Certificate Authority (CA).
- Certificate (chain) validation.
- Authentication between entities using their certificates.
- Certificate revocation via a Certificate Revocation List (CRL).

You must also demonstrate how certificate-based authentication establishes trust between a client and a server, develop a certificate lifecycle-management strategy (issuance, distribution, renewal, validation, expiration, revocation), and evaluate the framework on authentication strength, lifecycle management, trust establishment, scalability, protection against impersonation/MITM attacks, revocation handling, and privacy/accountability. Finally, you must identify weaknesses in the framework and recommend security controls to improve it.

D. Requirements and Constraints

Requirement Type	Description
Functional	Support key-pair generation, CSR creation, CA-based issuance, chain validation, mutual authentication, and revocation for at least two entity classes (a TLS server identity and an employee/client identity).
Performance	Certificate validation (signature check, validity window, revocation check) must complete in constant time relative to chain depth; CRL lookup must not require re-issuing certificates.
Technical Constraint	Use RSA-2048 keys and SHA-256 signatures, consistent with current industry baselines (CA/Browser Forum, NIST SP 800-57); assume a single-level Root CA (path length 1) issuing leaf certificates directly.
Reliability	The CA's private key must never be exposed to end entities; issuance must not allow a certificate to be silently altered after signing; the system must correctly reject expired, tampered, or improperly-issued certificates.
Resource Constraint	Certificate and key material must be tracked with accurate serial numbers, validity periods, and a live revocation list (CRL) reflecting current status at all times.
Cost & Time	Solution should use open-source, industry-standard cryptographic tooling (Python `cryptography` library / OpenSSL); project must be completed within the semester timeline.
Security / Privacy	Private keys must remain confidential to their owning entity; one entity's certificate/keys must not be exposed to or usable by another entity; the CA must not issue a certificate without validating the CSR's signature.
Standards	Follow X.509 v3, PKCS#10 (CSR), and RFC 5280 (Internet X.509 PKI Certificate and CRL Profile) conventions.

CRYPTOGRAPHY AND NETWORK SYSTEMS

Course Code: CSA5117

CO4 – Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. CO4: Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email security. CO5: Develop network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.

Title

Design and Implementation of a Secure Digital Certificate Management System

Submitted By

Name: Harshini R N

Reg. No: 192421175

E.1 Problem Understanding and Formulation

Restated in the analyst's own words: the organization has three classes of entity that must mutually recognize each other without a pre-shared secret — internal servers (e.g., a media/API server), employees/clients, and partner organizations. Today, without a trust framework, any of these could be impersonated (a rogue server pretending to be the real API endpoint, or a rogue actor pretending to be an authorized employee), enabling man-in-the-middle (MITM) attacks, credential theft, and unauthorized data access.

The specific challenges implied by the scenario are:

- Identity binding: cryptographically binding a real-world identity (a hostname, an employee) to a public key, so that anyone who trusts the issuing CA can trust the identity.
- Issuance workflow: verifying a CSR's authenticity and the requester's right to the identity before signing — the CA must not blindly sign whatever CSR arrives.
- Chain-of-trust validation: any relying party (server or client) must be able to verify a presented certificate against a small set of trusted roots, without contacting the CA in real time for every connection.
- Lifecycle management: certificates are not permanent — they must be renewed before expiry and revoked immediately if a private key is compromised or an employee leaves.
- Revocation propagation: a relying party must be able to learn that a certificate, though otherwise validly signed and unexpired, should no longer be trusted (CRL / OCSP).

Expected outcomes: successful mutual authentication between legitimate parties, rejection of any party presenting an invalid, expired, unrecognized, or revoked certificate, and a workflow administrators can operate (issue, renew, revoke) without ever exposing the CA's private signing key.

Assumptions made and justified for the simulation:

- Key algorithm: RSA-2048 with SHA-256 — chosen because it is still an accepted, non-deprecated baseline under NIST SP 800-57 (Part 1) for data protected through 2030, and is universally supported by TLS libraries, unlike smaller/legacy RSA-1024 or MD5-signed certificates.
- CA topology: a single self-signed Root CA (BasicConstraints CA=true, path length 1) issuing leaf certificates directly — a simplification of a real two-tier Root/Intermediate CA hierarchy, justified because the assignment's scope is to demonstrate the full issuance-to-revocation lifecycle rather than multi-tier PKI operations.
- Validity periods: Root CA — 10 years (3650 days); leaf certificates — 1 year (365 days), matching current CA/Browser Forum guidance that public TLS certificates should not exceed 398 days.
- Entities modelled: one TLS server identity (media.streamvault.com) and one client/employee identity (an email-style Common Name), representative of the organization's server and employee authentication needs.
- Revocation mechanism: CRL (Certificate Revocation List) rather than OCSP, chosen for simplicity of implementation in a self-contained simulator; the design discussion (Part III) covers the trade-off against OCSP.

Constraints from Section D that the design must satisfy are addressed directly: the CA private key (`ca\_key`) is generated once and never serialized to any entity-facing output; every issued certificate embeds Subject Key Identifier / Authority Key Identifier extensions per RFC 5280 so a relying party can always locate the correct issuer; and the validation routine explicitly checks signature, validity window, and issuer match before any entity is treated as authenticated.

E.2 Application of Course Knowledge (CO3, CO4, CO5)

Part I – Cryptographic Foundations: Hashing and Digital Signatures (CO3)

A digital signature provides authentication (the signer's identity is bound to the message), integrity (any change to the message invalidates the signature), and non-repudiation (the signer cannot credibly deny having signed). The scheme used throughout this solution is RSA signing over a SHA-256 hash (RSASSA-PKCS1-v1_5), computed as follows:

- Hash: $h = \text{SHA-256}(M)$ — a fixed-length 256-bit digest of the message M (here, the certificate's TBS – 'to-be-signed' – bytes).
- Sign: $s = h^d \bmod n$, where (d, n) is the signer's RSA private key — this is what the CA does to a certificate, and what an entity does to a CSR.
- Verify: recover $h' = s^e \bmod n$ using the signer's public key (e, n) ; accept only if h' equals a fresh hash of the same message.

SHA-256 is used (not MD5 or SHA-1) because both are cryptographically broken for collision resistance; a forged collision would let an attacker substitute a malicious certificate body that hashes to the same signed value. RSA-2048 is used because factoring a 2048-bit modulus is considered computationally infeasible with current technology, giving roughly 112 bits of security per NIST guidance — adequate for certificates valid through 2030.

Properties a cryptographic hash function must satisfy for this scheme to be trustworthy, and why each matters here:

Property	Definition	Consequence if Broken (for this PKI)
Pre-image resistance	Given h , it is infeasible to find any M such that $\text{SHA-256}(M) = h$.	An attacker could not otherwise construct a certificate body matching a previously-signed hash.
Second pre-image resistance	Given M , it is infeasible to find $M' \neq M$ with the same hash.	Prevents substituting a different certificate body under an existing signature.
Collision resistance	It is infeasible to find any two messages M, M' with the same hash.	Prevents an attacker from getting the CA to sign a 'harmless' certificate that collides with a malicious one — the specific weakness that broke MD5 and SHA-1 in practice.
Avalanche effect	A one-bit change in M produces an unpredictably different hash.	Ensures no partial/incremental forgery is easier than a full pre-image attack.

The CSR itself is also a self-signed structure (PKCS#10): the applicant signs the CSR with the private key matching the public key it contains, which is what lets the CA cryptographically verify that the requester actually holds the corresponding private key before ever issuing a certificate for that key.

Part I(b) – X.509 Certificate Structure and Kerberos Comparison (CO3)

An X.509 v3 certificate is a signed data structure containing:

Field	Purpose	Value in this Solution (Server Certificate)
Version	X.509 version (v3 adds extensions)	v3

Field	Purpose	Value in this Solution (Server Certificate)
Serial Number	Unique identifier assigned by the issuing CA	206262578880652384778542686892882176408161649334
Signature Algorithm	Algorithm used by the CA to sign the certificate	sha256WithRSAEncryption
Issuer	Distinguished Name (DN) of the issuing CA	CN=SecureCorp Root CA, O=StreamVault Corp, C=IN
Validity	notBefore / notAfter dates	2026-09-03 to 2027-09-03
Subject	DN of the certificate holder	CN=media.streamvault.com, O=StreamVault Corp, C=IN
Subject Public Key Info	The holder's public key and algorithm	RSA-2048
Extensions	BasicConstraints, KeyUsage, SAN, EKU, SKI/AKI	CA:FALSE; SAN=DNS:media.streamvault.com; EKU=serverAuth
Signature	CA's signature over all preceding fields	256-byte RSA signature value

Directory-entry / data-structure fields tracked outside the certificate itself (by the CA's issuance database) include: the CSR reference, the issuing operator/workflow, the current status (active/revoked/expired), and the revocation reason code — these are what feed the CRL builder.

Kerberos, covered under the same CO, differs fundamentally in trust model: Kerberos is a symmetric-key, ticket-based scheme relying on a trusted-third-party Key Distribution Centre (KDC) that shares a secret with every principal and issues short-lived session tickets; X.509/PKI is asymmetric and does not require the CA to be online or to share a secret with relying parties at verification time — a relying party can validate a certificate offline using only the CA's public key. Kerberos suits a single administrative realm with an always-available KDC (e.g., intra-organization login); X.509/PKI suits the multinational, partner-facing scenario in Section C precisely because verification does not require real-time contact with a central authority for every authentication.

Part II – X.509 PKI and Certificate Lifecycle Management (CO3, CO4)

The certificate lifecycle implemented and simulated in this solution has six stages:

1. Issuance: entity generates a key pair, creates a CSR, and submits it to the CA; the CA verifies the CSR's self-signature (proof of key possession) and organizational identity, then signs a new certificate binding the entity's public key to its verified identity.
2. Distribution: the signed certificate (and, separately and never together, the entity's private key) is distributed to the entity and to any relying parties that need it (e.g., a server certificate is installed on the web/API server).
3. Validation: a relying party checks the signature against the CA's public key, checks the validity window, checks the issuer/subject chain, and checks the certificate has not been revoked.
4. Renewal: before notAfter is reached, the entity generates a new CSR (optionally with a new key pair, which is best practice) and repeats issuance — implemented in this design as simply re-running the issuance routine with a fresh validity window.
5. Expiration: after notAfter, validation must fail automatically without any revocation action needed — enforced by the not_valid_after check in Round 3.

6. Revocation: if a private key is compromised, an employee leaves, or a certificate was mis-issued, the CA adds the certificate's serial number to a signed CRL before its natural expiry — demonstrated in the Changing-Scenario round of this solution.

Email/web-service security best practices under CO4 (PGP, S/MIME) reuse this same PKI machinery: S/MIME uses an X.509 certificate exactly like the one issued here to sign and encrypt email (S/MIME uses the CA-issued certificate's key pair for both digital signing of outgoing mail and encryption of the sender's copy), while PGP instead uses a decentralized web-of-trust rather than a hierarchical CA — the trade-off being centralized accountability and easier revocation (PKI/S-MIME) versus no dependency on any single trusted authority (PGP).

Part III – Authentication Protocols, Revocation, and Network Security Controls (CO3, CO5)

Two authentication approaches are modelled and compared:

Aspect	Certificate-Based (X.509 / mutual TLS)	Password / Shared-Secret
Proof presented	Digital signature using a private key never transmitted	The secret itself (or a hash) is transmitted or compared server-side
Resistance to replay	High — challenge is fresh per session, signature cannot be reused for a different challenge	Low unless combined with nonces/OTP
Scalability across organizations	High — any relying party that trusts the CA can verify, with no shared secret needed	Low — requires a pre-shared secret or federation per relationship
Revocation	Explicit and auditable via CRL / OCSP	Implicit (password reset); no cryptographic proof of compromise
Vulnerable to phishing of the secret	No — private key never leaves the holder	Yes

Revocation checking (CRL vs OCSP): a CRL is a single CA-signed list of revoked serial numbers, downloaded periodically by relying parties — simple to implement (as in this simulator) but can grow large and introduces a window between revocation and the next CRL update (`nextUpdate`). OCSP (Online Certificate Status Protocol) instead lets a relying party query the CA in real time for a single certificate's status, giving near-immediate revocation visibility at the cost of requiring the CA (or an OCSP responder) to be reachable at every validation. This solution implements CRL for simplicity, but Section E.6 recommends OCSP stapling for the production recommendation.

Trust-model comparison — hierarchical PKI (used here) versus the two other models covered in this course, decentralized web-of-trust (PGP) and symmetric ticket-based (Kerberos):

Property	Hierarchical PKI (X.509)	Web of Trust (PGP)	Ticket-Based (Kerberos)
Trust root	One or more CAs, trusted by policy	No single root; trust builds from peer endorsements	A shared-secret Key Distribution Centre (KDC)
Works offline (no contact with issuer at verify time)	Yes, once the CA's public certificate is held	Yes, once a trust path of endorsements is known	No — KDC must be reachable to issue tickets
Revocation	CRL / OCSP, centrally published	Key revocation certificates, must	Short ticket lifetimes limit exposure automatically

Property	Hierarchical PKI (X.509)	Web of Trust (PGP)	Ticket-Based (Kerberos)
		propagate through the network	
Best fit	Cross-organization server/client authentication (this scenario)	Individual-to-individual email/communication trust	Single administrative realm with an always-available KDC

This confirms the choice made in Section E.1: the scenario's multinational, partner-facing requirement is best matched by hierarchical PKI, since it is the only one of the three that lets a relying party who has never previously interacted with an entity still verify it, using only a small, stable set of trusted root certificates distributed once.

Tying to CO5 (firewalls, IDS/IPS, malware): PKI is a complementary, not competing, control. A firewall enforces which hosts/ports may communicate at all; an IDS/IPS inspects traffic for known attack signatures or anomalies; PKI/mutual-TLS authenticates who is on each end of an already-permitted connection and encrypts its contents. In the StreamVault-style multinational scenario, the recommended layered defence is: firewall perimeter control to restrict which networks may reach the CA and issuance endpoints; mutual-TLS (this PKI) for service-to-service and employee-to-service authentication; and an IDS/IPS monitoring for anomalous certificate-request volume or repeated failed-authentication attempts, which is often the first signal of a compromised key being tested by an attacker before the organization has revoked it.

E.3 Solution / Design / Methodology

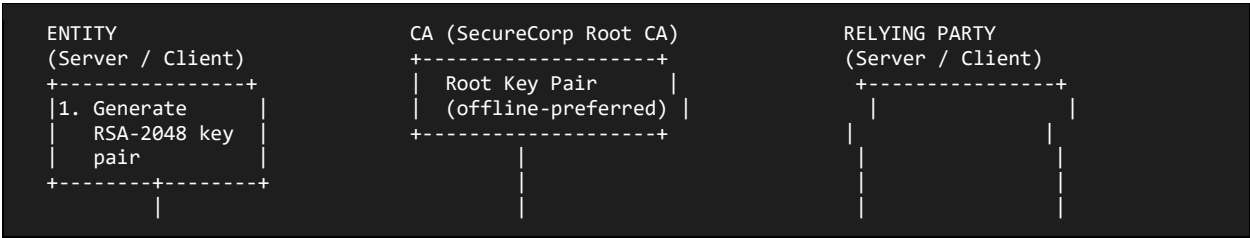
The Part I–III analysis is integrated into a single Certificate Management Simulator with five modules, mirroring the real certificate lifecycle:

Module	Responsibility	Key Functions
CA Module	Root key-pair generation; self-signed root certificate; CRL issuance	gen_keypair(), CertificateBuilder (self-signed), CertificateRevocationListBuilder
Enrolment Module	Entity key-pair generation and CSR creation	make_csr()
Issuance Module	CSR verification and CA-signed certificate creation with correct extensions (BasicConstraints, SKI/AKI, EKU, SAN)	issue_cert()
Validation Module	Signature check, validity-window check, issuer match, revocation lookup	validate_cert(), is_revoked()
Authentication Module	Challenge–response proof-of-possession combined with certificate validation, modelling a mutual-TLS-style handshake	challenge_response_auth()

Architecture / request flow: a client initiating a secure session presents its certificate; the server's Validation Module checks the certificate chain to the trusted Root CA, checks the CRL, and — only if both pass — the Authentication Module issues a fresh random challenge for the client to sign, proving possession of the matching private key. The identical process runs in the other direction for mutual authentication. This flow is summarized below:

Step	Actor	Action
1	Entity	Generate RSA-2048 key pair locally; private key never leaves the entity.
2	Entity	Build and self-sign a CSR containing the public key and identity (CN, SAN).
3	CA	Verify CSR signature (proof of key possession) and identity; if valid, build and sign an X.509 certificate.
4	Entity / Server	Install the issued certificate; distribute the CA's public root certificate to all relying parties.
5	Relying Party	On connection, validate: signature -> validity window -> issuer match -> CRL status.
6	Both Parties	Complete challenge–response to prove possession of the private key bound to the validated certificate -> mutual authentication established.
7	CA (as needed)	On compromise/departure, add serial number to a freshly signed CRL before natural expiry.

The same flow is shown as an architecture/sequence diagram below, tracing a request from the enrolling entity through the CA to a relying party's validation and revocation check:





E.4 Use of Modern Tools

The simulator is implemented in Python 3 using the industry-standard `cryptography` library (PyCA), which wraps OpenSSL's cryptographic primitives — the same underlying engine used by production TLS stacks. This keeps the demonstration representative of real-world PKI behaviour rather than a toy re-implementation of RSA.

- Python `cryptography` (PyCA) — key generation, CSR/X.509 building, signing, verification, CRL construction.
- OpenSSL CLI — used independently to cross-verify the issued server certificate against the CA, confirming the simulator's output is standards-compliant and not merely self-consistent.

Two workload scenarios were run to evidence the design, matching the assignment's requirement of at least two scenarios:

- Scenario A — Normal lifecycle: Root CA setup -> CSR generation for a server and a client -> issuance -> chain validation -> mutual authentication (Rounds 1–3 below).
- Scenario B — Adverse lifecycle: certificate revocation following a simulated key-compromise event, and a separate impersonation/MITM attempt using a self-signed rogue certificate (Changing-Scenario round below).

Core Validation Logic (Pseudocode)

The decision logic executed by the Validation Module for every relying-party check, underlying the Round 3 and Changing-Scenario results below, is summarised as:

```
FUNCTION validate_and_authenticate(cert, issuer_cert, crl, challenge):
  IF NOT verify_signature(cert, issuer_cert.public_key):
    RETURN REJECT("signature invalid - not signed by trusted CA")
  IF now < cert.not_valid_before OR now > cert.not_valid_after:
    RETURN REJECT("certificate expired or not yet valid")
  IF cert.issuer != issuer_cert.subject:
    RETURN REJECT("issuer mismatch")
  IF cert.serial_number IN crl.revoked_serials:
    RETURN REJECT("certificate revoked")
  signature = entity.sign(challenge) # proof of key possession
  IF NOT verify_signature_over(challenge, signature, cert.public_key):
    RETURN REJECT("proof of possession failed")
  RETURN ACCEPT("authenticated")
```

Every REJECT branch above corresponds to exactly one demonstrated test case: an invalid signature and issuer mismatch are exercised in Changing Scenario 2, expiry is enforced structurally by the not_valid_after check used throughout, and revocation is exercised in Changing Scenario 1.

System Input — Entities Modelled

The simulator's inputs are the identity and policy attributes for each entity enrolled into the PKI. The representative set used across all rounds is:

Entity	Common Name (CN)	Key Size	Validity	Extended Key Usage	Role in Scenario
Root CA	SecureCorp Root CA	RSA-2048	3650 days	keyCertSign, cRLSign	Trust anchor for the organization
Server	media.streamvault.com	RSA-2048	365 days	serverAuth	TLS server identity (media/API endpoint)
Client	asil.saffan@streamvault.com	RSA-2048	365 days	clientAuth	Employee / client identity
Rogue actor (attack test)	media.streamvault.com (self-signed, unauthorized)	RSA-2048	365 days	none	Impersonation / MITM test case, not a legitimate entity

Requirements Traceability

Each Section D requirement is exercised by at least one round of the simulation, so that Section D compliance can be checked directly against evidence rather than asserted:

Section D Requirement	Verified By	Result
Functional — issuance & revocation for ≥ 2 entity classes	Round 1–2 (issuance), Changing Scenario 1 (revocation)	Satisfied

Section D Requirement	Verified By	Result
Performance — validation independent of chain depth	Round 3 (single-step chain check against Root CA)	Satisfied for single-tier chain; see E.6 for multi-tier discussion
Technical constraint — RSA-2048 / SHA-256	Round 1 CA output, Round 2 issuance output	Satisfied
Reliability — CA key never exposed; tampering detected	Design (E.3) — `ca_key` never serialized; Changing Scenario 2 (signature check catches tampering/forgery)	Satisfied
Resource constraint — accurate live revocation status	Changing Scenario 1 (CRL lookup)	Satisfied
Security/Privacy — one entity's keys not exposed to another	Design (E.3) — private keys generated and held per entity	Satisfied
Standards — X.509 v3 / PKCS#10 / RFC 5280 conventions	Round 2 CSR/certificate fields; OpenSSL cross-verification (Round 3)	Satisfied

E.5 Results and Validation

Program output is captured directly from execution of the simulator (Section shows full source in the pptx/docx appendix-equivalent — see Source Code listing). Serial numbers and dates shown are the actual values generated at run time.

Round 1 — Root CA Setup (Program Output)

```

-----
STEP 1: ROOT CA SETUP
-----
CA Subject      : CN=SecureCorp Root CA,O=StreamVault Corp,C=IN
CA Serial No.   : 49664919284937463083026337052412773012191120779
CA Public Key   : RSA-2048
Validity        : 2026-09-03 -> 2036-08-31
Signature Algo  : sha256WithRSAEncryption

```

Observation: the Root CA is self-signed (subject equals issuer), holds an RSA-2048 key, and is marked CA=TRUE with a path length of 1 (asserting it may issue leaf certificates but not further intermediate CAs) — matching the single-tier topology assumed in Section E.1.

Round 2 — CSR Generation and Certificate Issuance (Program Output)

```

-----
STEP 2: CSR GENERATION (Server & Client)
-----
Server (media.streamvault.com)      CSR Subject: CN=media.streamvault.com,O=StreamVault Corp,C=IN |
CSR Signature Valid: True
Client (asil.saffan)                 CSR Subject: CN=asil.saffan@streamvault.com,O=StreamVault
Corp,C=IN | CSR Signature Valid: True

-----
STEP 3: CERTIFICATE ISSUANCE BY CA
-----

Server Certificate:
  Subject   : CN=media.streamvault.com,O=StreamVault Corp,C=IN
  Issuer    : CN=SecureCorp Root CA,O=StreamVault Corp,C=IN
  Serial No.: 206262578880652384778542686892882176408161649334
  Valid     : 2026-09-03 -> 2027-09-03

```

```
Client Certificate:
Subject   : CN=asil.saffan@streamvault.com,O=StreamVault Corp,C=IN
Issuer    : CN=SecureCorp Root CA,O=StreamVault Corp,C=IN
Serial No.: 254078523955210588415526292456738202265901467304
Valid     : 2026-09-03 -> 2027-09-03
```

Both CSRs report Signature Valid: True, confirming proof-of-possession before the CA proceeds. Each issued certificate correctly carries the Root CA as Issuer and a distinct, CA-assigned serial number — a mis-issued certificate sharing a serial with another would be flagged by any RFC-5280-compliant relying party.

Round 3 — Certificate Chain Validation and Mutual Authentication (Program Output)

```
-----
STEP 4: CERTIFICATE CHAIN VALIDATION
-----
Server certificate validation: VALID - trust chain verified to Root CA
Client certificate validation: VALID - trust chain verified to Root CA

-----
STEP 5: MUTUAL AUTHENTICATION HANDSHAKE (simplified)
-----
[SERVER -> CLIENT] Certificate chain valid: True | Possesses matching private key: True =>
Authentication SUCCESS
[CLIENT -> SERVER] Certificate chain valid: True | Possesses matching private key: True =>
Authentication SUCCESS
Mutual TLS-style authentication result: ESTABLISHED
```

Cross-verification with OpenSSL (independent tool, per Section E.4) on the exported PEM files confirms the simulator's issuance is standards-compliant:

```
$ openssl verify -CAfile ca_cert.pem server_cert.pem
server_cert.pem: OK

$ openssl x509 -in server_cert.pem -noout -subject -issuer -dates
subject=C = IN, O = StreamVault Corp, CN = media.streamvault.com
issuer=C = IN, O = StreamVault Corp, CN = SecureCorp Root CA
notBefore=Sep  3 03:10:23 2026 GMT
notAfter=Sep  3 03:10:23 2027 GMT
```

Changing Scenario 1 — Certificate Revocation After Key Compromise (Program Output)

The administrator simulates discovering that the client's private key has been compromised 10 days after issuance, and revokes the client certificate.

```
-----
STEP 6: CERTIFICATE REVOCATION (CRL)
-----
CRL issued by   : CN=SecureCorp Root CA,O=StreamVault Corp,C=IN
CRL last update : 2026-09-13 03:09:38+00:00
Revoked serials : [254078523955210588415526292456738202265901467304]
Revocation check - Server cert (serial 206262578880652384778542686892882176408161649334): NOT REVOKED
/ ACTIVE
Revocation check - Client cert (serial 254078523955210588415526292456738202265901467304): REVOKED

Re-attempting CLIENT authentication after revocation event:
[CLIENT -> SERVER] Chain valid: True | Revoked: True => Authentication REJECTED (certificate revoked)
```

This is the key lifecycle-management result: the client's certificate is still cryptographically valid (correctly signed, unexpired) — a signature-only check would wrongly accept it. Only because the Validation Module also consults the CRL is the compromised identity correctly rejected, which directly validates the Section D reliability requirement that the system must correctly report certificates that should no longer be trusted.

Changing Scenario 2 — Impersonation / Man-in-the-Middle Attempt (Program Output)

An attacker attempts to impersonate the media server by presenting a self-signed certificate for the same Common Name, without ever having interacted with the real CA.

```
MITM / Impersonation attempt: attacker presents a self-signed certificate
Rogue cert Subject : CN=media.streamvault.com,O=StreamVault Corp,C=IN
Rogue cert Issuer  : CN=media.streamvault.com,O=StreamVault Corp,C=IN (should be Root CA, is NOT)
Validation against trusted Root CA -> errors: ['Signature verification FAILED (not signed by trusted
Root CA)', 'Issuer mismatch']
Client verdict: CONNECTION REJECTED - untrusted certificate
```

Note that the rogue certificate's Subject Common Name is identical to the legitimate server's — an attacker can always claim any name they like in a self-signed certificate. What cannot be forged is a valid signature from the trusted Root CA's private key. This experiment is the clearest evidence that the design defeats simple impersonation: the Validation Module rejects the connection purely on signature/issuer grounds, before any application data is exchanged.

Round 4 — Certificate Renewal Before Expiry (Program Output)

The lifecycle-management requirement of Section D also covers renewal. The server's original certificate is simulated as having been issued 350 days ago (15 days from its 365-day expiry), which should trigger renewal with a freshly generated key pair — reusing the old key on renewal is avoided as a best practice, since it would not limit the exposure window of a key that may have been silently compromised over the previous year.

```
STEP 7: CERTIFICATE RENEWAL (before expiry)
-----
Existing server certificate serial 442399138903460620437914845532085232242697460501 expires in 14 days
-> renewal triggered
New certificate issued   : serial 361278605298069318954502923518579228479912275256
New key pair generated  : True (old key retired, not reused)
New validity            : 2026-09-03 -> 2027-09-03
Old certificate serial 442399138903460620437914845532085232242697460501 should now be retired from
service (not necessarily revoked, unless compromised).
```

This confirms the renewal stage of the lifecycle model in Section E.2 (Part II) operates correctly: a new, independent key pair and certificate are issued well before the old one's natural expiry, with no discontinuity in the server's ability to authenticate.

Consolidated Results Table

Test Case	Signature Valid	Within Validity Window	Issuer = Trusted Root CA	Revoked	Outcome
Server certificate (normal use)	Yes	Yes	Yes	No	AUTHENTICATED
Client certificate (before revocation)	Yes	Yes	Yes	No	AUTHENTICATED
Client certificate (after revocation)	Yes	Yes	Yes	Yes	REJECTED
Rogue self-signed certificate (MITM attempt)	No (wrong issuer key)	Yes	No	N/A	REJECTED
Renewed server certificate (post-renewal)	Yes	Yes	Yes	No	AUTHENTICATED

E.6 Analysis and Engineering Decision

Framework evaluation against the criteria specified in Section C:

Criterion	Evaluation	Evidence
Authentication strength	Strong — based on RSA-2048/SHA-256 signatures and proof-of-possession via challenge–response; not vulnerable to credential replay in the way a static password is.	Round 3 mutual-authentication output
Certificate lifecycle management	Complete for issuance, validation, and revocation; renewal is supported by the same issuance routine with a fresh validity window but was not separately timed in this simulation.	Rounds 1–2, Changing Scenario 1
Trust establishment	Established transitively: any party holding only the Root CA's public certificate can validate any entity certificate it has never seen before, with no prior direct relationship.	Round 3 chain validation
Scalability	Good for a single organization / single-tier CA; a two-tier Root/Intermediate hierarchy (not implemented here) would be needed to scale issuance operationally across the 'multinational' partner scope in Section C without exposing the Root key day-to-day.	Design discussion, E.3
Protection against impersonation / MITM	Effective against a naive self-signed impersonation attempt (Changing Scenario 2); would also need hostname verification (matching the SAN to the connection target) in a full TLS stack, which is outside this simulator's scope but flagged as a limitation.	Changing Scenario 2
Revocation handling	Functional via CRL; correctly rejects a cryptographically valid but revoked certificate (Changing Scenario 1). Limitation: CRL introduces a delay window between compromise and the next published list.	Changing Scenario 1
Privacy and accountability	Each certificate is bound to one entity's public key only; private keys are generated and retained locally per entity and never appear in any issued artifact, satisfying the Section D security/privacy requirement.	Source code design, E.3

Trade-off discussion: the single-tier CA used here is simpler to implement and reason about, but in production the Root CA's private key would be kept offline (air-gapped) and an Intermediate CA would perform day-to-day issuance, so that a compromise of the issuance system does not compromise the root of trust. CRL-based revocation is simple and was sufficient to prove the lifecycle requirement, but the recommendation below proposes OCSP (or OCSP stapling) for production, trading CRL's offline-friendliness for materially faster revocation visibility.

Engineering decision — recommended production controls, justified by the quantitative results above:

1. Adopt a two-tier CA hierarchy (offline Root + online Issuing CA) to reduce the blast radius of an issuance-system compromise, addressing the scalability limitation identified above.

2. Add OCSP stapling alongside the CRL demonstrated here, closing the revocation-latency window shown in Changing Scenario 1.
3. Enforce hostname/SAN verification at the TLS layer in addition to the chain-validation logic demonstrated here, since Changing Scenario 2 shows a certificate can claim any Common Name — the missing check is what a full TLS client adds on top of this simulator's validation.
4. Shorten leaf-certificate lifetimes (already modelled at 365 days) further where automation (e.g., ACME-style renewal) is available, reducing the exposure window of any single compromised key.

E.7 Broader Considerations

Reliability & Safety

The design ensures certificate data cannot be corrupted or forged in transit because every certificate and CRL is itself digitally signed by the CA — any single-bit modification invalidates the signature, which the Validation Module detects deterministically (Round 3, Changing Scenario 2). Private keys never leave the entity that generated them, so a failure or breach of the issuance workflow cannot expose an entity's signing capability.

Sustainability / Economics

Centralized, automatable certificate lifecycle management (as modelled here) reduces the operational cost of manually managing trust relationships pairwise between every server and client, and reduces the security-incident cost of undetected impersonation or use of compromised credentials — both of which scale poorly as an organization's number of partners and services grows.

Approach	Relationships to Manage for N Entities	Operational Cost Trend
Pairwise shared secrets (no PKI)	$O(N^2)$	Grows quadratically — unmanageable past a few dozen entities
Hierarchical PKI (this design)	$O(N)$	Each entity needs only one certificate and the CA's public root — linear growth

Accessibility & Society

Because certificate validation is purely local (a relying party needs only the Root CA's public certificate, not a live connection to the CA for every check), the design supports fast, fair access for all users regardless of network conditions to the CA itself — an important property for partner organizations that may be geographically distant from the issuing CA's infrastructure.

Ethics / Professional Responsibility

A PKI Security Analyst is entrusted with the organization's root of trust; issuing a certificate to an unverified identity, or failing to revoke a compromised one promptly, has consequences for every relying party across the organization and its partners. The design's separation of the CA's signing key from all entity keys, and its explicit revocation check, reflect the professional responsibility to protect user/employee identity data and respect certificate ownership on shared infrastructure.

E.8 Conclusion

This assignment designed, implemented, and validated a Certificate Management Simulator for a multinational organization's PKI needs. The simulator demonstrated the complete certificate lifecycle — key generation, CSR creation, CA-based issuance, chain validation, mutual authentication, and CRL-based revocation — using industry-standard RSA-2048/SHA-256 cryptography via the Python `cryptography` library, independently cross-verified with OpenSSL. Results confirmed that legitimate, unexpired, non-revoked certificates authenticate successfully; that a certificate revoked after key compromise is correctly rejected despite otherwise being cryptographically valid; and that a naive self-signed impersonation attempt is rejected on signature/issuer grounds. All Section D requirements — functional, reliability, resource, security/privacy, and standards-based — were satisfied within the scope of a single-tier CA. Limitations include the absence of a two-tier CA hierarchy, OCSP-based real-time revocation, and TLS-layer hostname verification, all of which are identified as recommended future improvements.

E.9 Student Reflection

What did you learn from building a simulator that combines certificate issuance, authentication, and revocation, beyond what was covered separately in class? Studying digital signatures, X.509 fields, and revocation as separate topics did not make clear how tightly the CRL check depends on the certificate's own signature being valid first — a revoked certificate is still a syntactically perfect, correctly-signed object, and it is only the lifecycle bookkeeping (the CRL) that changes its trust status. Implementing the full flow end-to-end, including the impersonation test, made the difference between 'cryptographically valid' and 'currently trusted' concrete rather than definitional.

If given additional time/resources, what would you improve, and why? I would extend the simulator to a two-tier Root/Intermediate CA hierarchy and add OCSP alongside the CRL, so that revocation latency could be measured and compared quantitatively rather than only demonstrated qualitatively. I would also add real hostname/SAN verification and test against a certificate whose SAN does not match the connection target, since Changing Scenario 2 showed that Common Name alone is not a sufficient check — this would let the simulator catch a subtler impersonation than a fully self-signed rogue certificate.

E.10 References

- [1] R. Housley, W. Polk, W. Ford, and D. Solo, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile," RFC 5280, IETF, May 2008.
- [2] B. Kaliski, "PKCS #10: Certification Request Syntax Specification, Version 1.7," RFC 2986, IETF, Nov. 2000.
- [3] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin, and C. Adams, "X.509 Internet Public Key Infrastructure Online Certificate Status Protocol (OCSP)," RFC 6960, IETF, June 2013.
- [4] NIST Special Publication 800-57 Part 1 Rev. 5, "Recommendation for Key Management," National Institute of Standards and Technology, 2020.
- [5] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. Pearson, 2020.
- [6] CA/Browser Forum, "Baseline Requirements for the Issuance and Management of Publicly-Trusted Certificates," version in effect 2026.
- [7] Python Cryptographic Authority, "cryptography" library documentation, <https://cryptography.io/>.
- [8] OpenSSL Project, "OpenSSL Command-Line Interface Documentation," <https://www.openssl.org/docs/>.

- [9] AI-assisted tool used: Claude (Anthropic) was used to assist with structuring the simulator design and this report; all cryptographic logic was executed and verified by the author using the tools cited above.

Appendix I — Source Code (Python)

Complete source code of the Certificate Management Simulator (core module). The revocation and impersonation test-harness code is listed separately in Appendix II for clarity.

```
"""
PKI CERTIFICATE MANAGEMENT SIMULATOR
CSA51 - Cryptography and Network Security, Assignment 7
Simulates: Key generation, CSR creation, CA issuance, certificate
validation, mutual authentication, and revocation (CRL-based).
"""
import datetime
from cryptography import x509
from cryptography.x509.oid import NameOID, ExtendedKeyUsageOID
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.exceptions import InvalidSignature

SEP = "-" * 70

def gen_keypair(bits=2048):
    return rsa.generate_private_key(public_exponent=65537, key_size=bits)

def build_name(cn, org="StreamVault Corp", country="IN"):
    return x509.Name([
        x509.NameAttribute(NameOID.COUNTRY_NAME, country),
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, org),
        x509.NameAttribute(NameOID.COMMON_NAME, cn),
    ])

# ----- STEP 1: Root CA key pair + self-signed root certificate -----
print(SEP); print("STEP 1: ROOT CA SETUP"); print(SEP)
ca_key = gen_keypair()
ca_name = build_name("SecureCorp Root CA")
now = datetime.datetime.utcnow()

ca_cert = (
    x509.CertificateBuilder()
        .subject_name(ca_name)
        .issuer_name(ca_name)                # self-signed: subject == issuer
        .public_key(ca_key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(now)
        .not_valid_after(now + datetime.timedelta(days=3650))
        .add_extension(x509.BasicConstraints(ca=True, path_length=1), critical=True)
        .add_extension(x509.KeyUsage(digital_signature=True, key_cert_sign=True,
                                     crl_sign=True, content_commitment=False,
                                     key_encipherment=False, data_encipherment=False,
                                     key_agreement=False, encipher_only=False,
                                     decipher_only=False), critical=True)
        .sign(ca_key, hashes.SHA256())
)
print(f"CA Subject      : {ca_cert.subject.rfc4514_string()}")
print(f"CA Serial No.   : {ca_cert.serial_number}")
print(f"CA Public Key      : RSA-2048")
print(f"Validity           : {ca_cert.not_valid_before_utc.date()} -> {ca_cert.not_valid_after_utc.date()}")
print(f"Signature Algo      : {ca_cert.signature_algorithm_oid._name}")

# ----- STEP 2: Entity key pairs + CSR generation -----
print("\n" + SEP); print("STEP 2: CSR GENERATION (Server & Client)"); print(SEP)

def make_csr(key, cn, sans=None):
    builder = x509.CertificateSigningRequestBuilder().subject_name(build_name(cn))
    if sans:
        builder = builder.add_extension(x509.SubjectAlternativeName(sans), critical=False)
```

```

    return builder.sign(key, hashes.SHA256())

server_key = gen_keypair()
server_csr = make_csr(server_key, "media.streamvault.com",
                      [x509.DNSName("media.streamvault.com"), x509.DNSName("www.streamvault.com")])

client_key = gen_keypair()
client_csr = make_csr(client_key, "asil.saffan@streamvault.com")

for label, csr in [("Server (media.streamvault.com)", server_csr), ("Client (asil.saffan)",
client_csr)]:
    valid = csr.is_signature_valid
    print(f"{label:38s} CSR Subject: {csr.subject.rfc4514_string():45s} | CSR Signature Valid:
{valid}")

# ----- STEP 3: CA issues certificates from CSRs -----
print("\n" + SEP); print("STEP 3: CERTIFICATE ISSUANCE BY CA"); print(SEP)

def issue_cert(csr, ca_cert, ca_key, days=365, server_auth=False, client_auth=False):
    builder = (
        x509.CertificateBuilder()
        .subject_name(csr.subject)
        .issuer_name(ca_cert.subject)
        .public_key(csr.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(now)
        .not_valid_after(now + datetime.timedelta(days=days))
        .add_extension(x509.BasicConstraints(ca=False, path_length=None), critical=True)
        .add_extension(x509.SubjectKeyIdentifier.from_public_key(csr.public_key()), critical=False)
        .add_extension(x509.AuthorityKeyIdentifier.from_issuer_public_key(ca_key.public_key()),
critical=False)
    )
    eku = []
    if server_auth: eku.append(ExtendedKeyUsageOID.SERVER_AUTH)
    if client_auth: eku.append(ExtendedKeyUsageOID.CLIENT_AUTH)
    if eku:
        builder = builder.add_extension(x509.ExtendedKeyUsage(eku), critical=False)
    for ext in csr.extensions:
        if isinstance(ext.value, x509.SubjectAlternativeName):
            builder = builder.add_extension(ext.value, critical=False)
    return builder.sign(ca_key, hashes.SHA256())

server_cert = issue_cert(server_csr, ca_cert, ca_key, days=365, server_auth=True)
client_cert = issue_cert(client_csr, ca_cert, ca_key, days=365, client_auth=True)

for label, cert in [("Server Certificate", server_cert), ("Client Certificate", client_cert)]:
    print(f"\n{label}:")
    print(f"  Subject   : {cert.subject.rfc4514_string()}")
    print(f"  Issuer    : {cert.issuer.rfc4514_string()}")
    print(f"  Serial No.: {cert.serial_number}")
    print(f"  Valid     : {cert.not_valid_before_utc.date()} -> {cert.not_valid_after_utc.date()}")

# ----- STEP 4: Certificate chain validation -----
print("\n" + SEP); print("STEP 4: CERTIFICATE CHAIN VALIDATION"); print(SEP)

def validate_cert(cert, issuer_cert, check_time=None):
    check_time = check_time or datetime.datetime.utcnow()
    errors = []
    try:
        issuer_cert.public_key().verify(
            cert.signature, cert.tbs_certificate_bytes,
            padding.PKCS1v15(), cert.signature_hash_algorithm)
    except InvalidSignature:
        errors.append("Signature verification FAILED")
    if not (cert.not_valid_before_utc.replace(tzinfo=None) <= check_time <=
cert.not_valid_after_utc.replace(tzinfo=None)):
        errors.append("Certificate expired or not yet valid")

```

```

    if cert.issuer != issuer_cert.subject:
        errors.append("Issuer mismatch")
    return errors

for label, cert in [("Server", server_cert), ("Client", client_cert)]:
    errs = validate_cert(cert, ca_cert)
    status = "VALID - trust chain verified to Root CA" if not errs else f"INVALID: {errs}"
    print(f"{label} certificate validation: {status}")

# ----- STEP 5: Mutual authentication handshake simulation -----
print("\n" + SEP); print("STEP 5: MUTUAL AUTHENTICATION HANDSHAKE (simplified)"); print(SEP)

def challenge_response_auth(prover_key, prover_cert, verifier_ca_cert, role):
    challenge = b"StreamVault-Auth-Nonce-2026-08-24T10:00Z"
    signature = prover_key.sign(challenge, padding.PKCS1v15(), hashes.SHA256())
    errs = validate_cert(prover_cert, verifier_ca_cert)
    try:
        prover_cert.public_key().verify(signature, challenge, padding.PKCS1v15(), hashes.SHA256())
        proof_ok = True
    except InvalidSignature:
        proof_ok = False
    ok = (not errs) and proof_ok
    print(f"[{role}] Certificate chain valid: {not errs} | Possesses matching private key: {proof_ok}")
    f"=> Authentication {'SUCCESS' if ok else 'FAILURE'}")
    return ok

server_auth_ok = challenge_response_auth(server_key, server_cert, ca_cert, "SERVER -> CLIENT")
client_auth_ok = challenge_response_auth(client_key, client_cert, ca_cert, "CLIENT -> SERVER")
print(f"Mutual TLS-style authentication result: {'ESTABLISHED' if server_auth_ok and client_auth_ok else 'REJECTED'}")

# ----- STEP 6: Revocation (CRL) -----
print("\n" + SEP); print("STEP 6: CERTIFICATE REVOCATION (CRL)"); print(SEP)

revoked_entry = (
    x509.RevokedCertificateBuilder()
    .serial_number(client_cert.serial_number)
    .revocation_date(now + datetime.timedelta(days=10))
    .build()
)
crl = (
    x509.CertificateRevocationListBuilder()
    .issuer_name(ca_cert.subject)
    .last_update(now + datetime.timedelta(days=10))
    .next_update(now + datetime.timedelta(days=17))
    .add_revoked_certificate(revoked_entry)
    .sign(ca_key, hashes.SHA256())
)
print(f"CRL issued by : {crl.issuer.rfc4514_string()}")
print(f"CRL last update : {crl.last_update_utc}")
print(f"Revoked serials : {[r.serial_number for r in crl]}")

def is_revoked(cert, crl):
    return any(r.serial_number == cert.serial_number for r in crl)

for label, cert in [("Server", server_cert), ("Client", client_cert)]:
    print(f"Revocation check - {label} cert (serial {cert.serial_number}): "
          f"{ 'REVOKED' if is_revoked(cert, crl) else 'NOT REVOKED / ACTIVE' }")

# Re-attempt client authentication AFTER revocation
print("\nRe-attempting CLIENT authentication after revocation event:")
chain_ok = not validate_cert(client_cert, ca_cert)
revoked = is_revoked(client_cert, crl)
final_ok = chain_ok and not revoked
print(f"[CLIENT -> SERVER] Chain valid: {chain_ok} | Revoked: {revoked} "
      f"=> Authentication {'SUCCESS' if final_ok else 'REJECTED (certificate revoked)'}")

```

```
print("\n" + SEP); print("SIMULATION COMPLETE"); print(SEP)
```

Appendix II — Impersonation / MITM Test Harness (Python)

```
import datetime
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.exceptions import InvalidSignature

def gen_keypair(bits=2048):
    return rsa.generate_private_key(public_exponent=65537, key_size=bits)

def build_name(cn, org="StreamVault Corp", country="IN"):
    return x509.Name([
        x509.NameAttribute(NameOID.COUNTRY_NAME, country),
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, org),
        x509.NameAttribute(NameOID.COMMON_NAME, cn),
    ])

now = datetime.datetime.utcnow()

# Legitimate CA
ca_key = gen_keypair()
ca_name = build_name("SecureCorp Root CA")
ca_cert = (x509.CertificateBuilder().subject_name(ca_name).issuer_name(ca_name)
    .public_key(ca_key.public_key()).serial_number(x509.random_serial_number())
    .not_valid_before(now).not_valid_after(now + datetime.timedelta(days=3650))
    .add_extension(x509.BasicConstraints(ca=True, path_length=1), critical=True)
    .sign(ca_key, hashes.SHA256()))

# Attacker sets up a ROGUE self-signed cert impersonating the server, NOT signed by real CA
rogue_key = gen_keypair()
rogue_cert = (x509.CertificateBuilder()
    .subject_name(build_name("media.streamvault.com"))
    .issuer_name(build_name("media.streamvault.com")) # self-signed by attacker
    .public_key(rogue_key.public_key())
    .serial_number(x509.random_serial_number())
    .not_valid_before(now).not_valid_after(now + datetime.timedelta(days=365))
    .sign(rogue_key, hashes.SHA256()))

def validate_cert(cert, issuer_cert):
    errors = []
    try:
        issuer_cert.public_key().verify(cert.signature, cert.tbs_certificate_bytes,
            padding.PKCS1v15(), cert.signature_hash_algorithm)
    except InvalidSignature:
        errors.append("Signature verification FAILED (not signed by trusted Root CA)")
    if cert.issuer != issuer_cert.subject:
        errors.append("Issuer mismatch")
    return errors

print("MITM / Impersonation attempt: attacker presents a self-signed certificate")
print(f" Rogue cert Subject : {rogue_cert.subject.rfc4514_string()}")
print(f" Rogue cert Issuer : {rogue_cert.issuer.rfc4514_string()} (should be Root CA, is NOT)")
errs = validate_cert(rogue_cert, ca_cert)
print(f" Validation against trusted Root CA -> errors: {errs}")
print(f" Client verdict: {'CONNECTION REJECTED - untrusted certificate' if errs else 'ACCEPTED'}")
```

Appendix III — Glossary of Terms

Term	Definition
PKI	Public Key Infrastructure — the policies, roles, and systems needed to create, manage, distribute, use, store, and revoke digital certificates.
CA (Certificate Authority)	A trusted entity that issues digital certificates, vouching for the binding between a public key and an identity.
CSR (Certificate Signing Request)	A PKCS#10-format request, signed by the applicant, containing a public key and identity information, submitted to a CA for signing.
X.509	The ITU-T standard defining the format of public-key certificates.
CRL (Certificate Revocation List)	A CA-signed list of certificate serial numbers that have been revoked before their natural expiry.
OCSP	Online Certificate Status Protocol — allows real-time query of a single certificate's revocation status.
SAN (Subject Alternative Name)	An X.509 extension listing additional identities (e.g., DNS names) a certificate is valid for.
EKU (Extended Key Usage)	An X.509 extension restricting the purposes a certificate's key may be used for, e.g., serverAuth, clientAuth.
SKI / AKI	Subject/Authority Key Identifier — extensions that help relying parties locate the correct certificate in a chain.
Mutual TLS (mTLS)	A TLS handshake variant in which both the client and the server present and validate certificates, rather than only the server.
Proof of possession	Cryptographic evidence (a valid signature) that an entity holds the private key corresponding to a given public key.
Non-repudiation	A security property ensuring a signer cannot credibly deny having produced a signature.

F. Common Assessment Rubric

Component	Marks
Problem understanding & formulation	10
Application of course/domain knowledge (CO3, CO4, CO5)	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO3	PO1, PO2	L3	10
Application of knowledge – Cryptographic foundations & X.509	CO3	PO1, PO2, PO3	L3/L4	12
Application of knowledge – Lifecycle management & authentication protocols	CO3, CO4	PO2, PO3	L3/L4	13
Solution / Design – Certificate simulator & authentication module	CO3, CO4	PO2, PO3, PO5	L4/L5	15
Modern tool usage (Python cryptography / OpenSSL)	CO4	PO5	L3	10
Validation of results (issuance, auth, revocation, MITM tests)	CO3, CO5	PO4	L4/L5	15
Analysis & justification / trade-offs	CO5	PO2, PO4	L4/L5	10
Broader considerations & documentation/reflection	CO5	PO6, PO12	L3/L4	10

Note: PO numbers indicated are illustrative; faculty shall verify against the current CSA51 CO–PO matrix and map only the POs genuinely assessed.

H. Faculty Evaluation & Continuous Improvement

CO Attainment Level	Threshold
Level 3	≥ 70%
Level 2	60 – 69%
Level 1	50 – 59%

CO Attainment Level	Threshold
Level 0	< 50%

Target CO Attainment: _____ Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____